

FELIX: A Ferroelectric FET Based Low Power Mixed-Signal In-Memory Architecture for DNN Acceleration

TAHA SOLIMAN, Robert Bosch GmbH

NELLIE LALENI, Fraunhofer IPMS, Center Nanoelectronic Technologies

TOBIAS KIRCHNER, Robert Bosch GmbH

FRANZ MÜLLER, ASHISH SHRIVASTAVA, and THOMAS KÄMPFE, Fraunhofer IPMS, Center Nanoelectronic Technologies

ANDRE GUNTORO, Robert Bosch GmbH

NORBERT WEHN, University of Kaiserslautern

Today, a large number of applications depend on deep neural networks (DNN) to process data and perform complicated tasks at restricted power and latency specifications. Therefore, processing-in-memory (PIM) platforms are actively explored as a promising approach to improve the throughput and the energy efficiency of DNN computing systems. Several PIM architectures adopt resistive non-volatile memories as their main unit to build crossbar-based accelerators for DNN inference. However, these structures suffer from several drawbacks such as reliability, low accuracy, large ADCs/DACs power consumption and area, high write energy, and so on. In this article, we present a new mixed-signal in-memory architecture based on the bit-decomposition of the multiply and accumulate (MAC) operations. Our in-memory inference architecture uses a single FeFET as a non-volatile memory cell. Compared to the prior work, this system architecture provides a high level of parallelism while using only 3-bit ADCs. Also, it eliminates the need for any DAC. In addition, we provide flexibility and a very high utilization efficiency even for varying tasks and loads. Simulations demonstrate that we outperform state-of-the-art efficiencies with 36.5 TOPS/W and can pack 2.05 TOPS with 8-bit activation and 4-bit weight precision in an area of 4.9 mm² using 22 nm FDSOI technology. Employing binary operation, we obtain 1169 TOPS/W and over 261 TOPS/W/mm² on system level.

CCS Concepts: • **Hardware** → **Analog and mixed-signal circuits**; • **Computer systems organization** → **Neural networks**;

Additional Key Words and Phrases: FeFET crossbar array, in-memory computation, convolution neural networks, mixed-signal processing, bit-decomposition, current-mode ADC

This work received funding within the ECSEL Joint Undertaking project TEMPO in collaboration with the European Union's H2020 Framework Program (H2020/2014-2020) and National Authorities, under grant agreement number 826655 and was partially funded by the Carl Zeiss Foundation under the grant Sustainable Embedded AI.

Authors' addresses: T. Soliman, T. Kirchner, and A. Guntoro, Robert Bosch GmbH, Renningen 71272, Germany; emails: {taha.soliman, tobias.kirchner, andre.guntoro}@de.bosch.com; N. Laleni, F. Müller, A. Shrivastava, and T. Kämpfe, Fraunhofer IPMS, Center Nanoelectronic Technologies, Dresden, Germany; emails: {nelli.laleni, franz.mueller, ashish.shrivastava, thomas.kaempfe}@ipms.fraunhofer.de; N. Wehn, University of Kaiserslautern, Kaiserslautern 67663, Germany; email: wehn@eit.uni-kl.de.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1539-9087/2022/10-ART84

<https://doi.org/10.1145/3529760>

ACM Transactions on Embedded Computing Systems, Vol. 21, No. 6, Article 84. Publication date: October 2022.

ACM Reference format:

Taha Soliman, Nellie Laleni, Tobias Kirchner, Franz Müller, Ashish Shrivastava, Thomas Kämpfe, Andre Gun-
toro, and Norbert Wehn. 2022. FELIX: A Ferroelectric FET Based Low Power Mixed-Signal In-Memory Archi-
tecture for DNN Acceleration. *ACM Trans. Embed. Comput. Syst.* 21, 6, Article 84 (October 2022), 25 pages.
<https://doi.org/10.1145/3529760>

1 INTRODUCTION

In recent years, **Deep Neural Networks (DNNs)** have made a huge leap in various complex tasks such as image recognition and natural language processing. However, these networks come at the cost of high computational and storage requirements. These requirements limit the usage of DNNs in various applications which suffer from restricted available computational resources and memory access. At the same time, the hardware resources needed by state-of-the-art DNNs keep growing to achieve better accuracy. This growth represents a problem that is observed in CMOS ASIC design. There, the on-chip memory is the biggest bottleneck for efficient energy computation and it can not even expand more to fulfill the requirements for such growing networks [22]. Additionally, in such systems, the usage of off-chip memory, results in both substantial energy and latency [47]. These challenging requirements lead to several research efforts focusing on solving the memory bottleneck and investigations for other possible computational paradigms such as near memory computing and in-memory computing.

Over the past years, several emerging non-volatile devices were presented as a solution to fulfill the aforementioned DNN requirements. These devices can perform logic and arithmetic operations while being the main storage cells of the in-memory computational systems. Several non-volatile technologies have been presented and utilized within these in-memory architectures, e.g., **resistive random-access memories (RRAM)** [46], **phase change memories (PCM)** [2], **magnetic tunnel junctions (MTJs)** [42], and **ferroelectric field-effect transistor (FeFET)** [32]. In these architectures, crossbars of **Non-Volatile-Memory (NVM)** units are built to perform **matrix-vector multiplication (MVM)** where DNN weights are stored within the memory cells. This topology substantially reduces the memory requirements related to data movement and the parallel nature of crossbars further accelerates DNN computations.

Despite the aforementioned advantages, the crossbar-based in-memory accelerators suffer from several drawbacks. First, NVM usually can not adopt high-precision computing due to the limited number of states that are available in an NVM cell [34, 43]. Second, as these accelerators perform operations in the analog domain, they are complemented by interfaces including **Analog to Digital Converters (ADCs)** and **Digital to Analog Converters (DACs)** to perform conversions from and into the digital domain. As the higher precision, these blocks become the more power consumption, latency, and area they need which compromises the whole system performance and makes it less efficient. For example, DACs/ADCs can constitute over 85% of the area and power consumption in crossbar-based systems [28].

Moreover, reducing the computations precision drops the accuracy of the inferred networks and limits the usage of these systems to small networks. Also, as the crossbar is restricted in size, mapping different weights of the networks' layers becomes challenging too. This can affect the overall system utilization since the mapping should maximize the data reuse as well as maintain high operational parallelization.

In our architecture, FeFETs are employed as the main memory cell since they offer several advantages over other embedded resistive memory concepts such as STT-MRAM, PCRAM, or OxRAM. FeFET as a storage transistor concept can act as current-source-like elements, making this implementation much less prone to various issues, e.g., IR drop or noise, which can be observed on

memristor based concepts. FeFETs with a memory window up to 2V exhibit a very high on/off current ratio exceeding 10^5 and the lowest programming power of about < 0.1 fJ in any embedded memory concept [5, 38]. The cells have a long-term retention of 10 years. Furthermore, long retention multi-bit performance has been demonstrated in FeFET [25]. Due to the nature of high-k **metal-oxide-semiconductor field-effect transistor (MOSFET)**, they also offer the smallest read-out latency of about 1 ns. FeFETs have so far been demonstrated in 28 nm bulk CMOS and 22 nm **fully depleted silicon on insulator (FDSOI)** technology but they offer a promising scaling roadmap towards smaller technology nodes, which distinguishes it from Flash-based concepts [19].

In this work, we tackle the drawbacks mentioned earlier by further optimizing the different system blocks as well as exploiting opportunities FeFET memory elements can offer in the digital and analog domains. We optimize the unit size in the crossbar array, requiring only 1 FeFET for each AND operation while insuring accessibility for both the inference and programming modes. Furthermore, we explore the bit-decomposition algorithm and map its operations to our different architecture blocks to accelerate the computation of DNNs and allow for various levels of parallelism.

In summary, our new contributions are the following:

- Novel bit-decomposition based mixed-signal architecture for the in-memory computation of **Convolutional Neural Networks (CNNs)** using FeFET as the memory cell which allows for small power consumption.
- Complete elimination of DACs while using lower-precision ADCs through efficient parallel bit decomposed MAC operations. Also, building current-based optimized ADC for our crossbar operations.
- Adopting completely flexible-precision which allows for a wide range of DNN and to trade accuracy for throughput and vice versa.
- Preserving system immunity against cell variability with accuracy loss of less than 2% at a highly optimized quantization and at high system utilization.
- Physical implementation in GLOBALFOUNDRIES technology and silicon verification of the core elements of the mixed-signal blocks by on-wafer characterization including corner states.
- We provide an in-depth system-level evaluation of FELIX architecture across various benchmarks. We combine the measurements from the taped-out system part as well as the simulation results to accurately benchmark the system performance.

Compared to our prior work [38, 39], we further optimized the crossbar structure and interface between the crossbar array and the mixed-signal blocks. In addition, we designed and implemented our own optimized current-based ADC to achieve better performance. Also, we performed a reorganization of the architecture components to further minimize the data transfer which includes showing the architecture pipeline and memory mapping. Finally, we extensively evaluate our architecture based on physical implementation measurements as well as simulations taking into consideration variability and system scaling at different target network precision.

Based on the performed evaluations, our system outperforms the state-of-the-art in-memory architectures using different memory technologies including FeFET while showing a high system utilization across various networks with different workloads.

This article is structured as follows: First, in Section 2, we go through related work and the current state-of-the-art. In Section 3, we provide a background on the basics of the FeFET crossbar design and the mode of operation as well as the convolutional neural networks bit-decomposition concept. In Section 4, we introduce our architecture where we breakdown the system into different components and elaborate the structure of each component individually as well as their mode of

operation. In Section 5, we show the system pipeline and the mapping of different kernels to the memory locations. Finally, we perform an extensive evaluation of our architecture using various benchmarks and at different modes of operations as well as a comparison to state-of-the-art systems.

2 RELATED WORK

Several recent in-memory architectures were presented to leverage the advances in NVM technologies. These architectures fall into one of three main categories.

In-memory analog-based architectures, several multi-precision analog-based crossbar structures were presented which maximize the usage of the analog properties the crossbar structure is offering. The analog crossbar is complemented with DACs to convert the digital input values to analog signals for the crossbar. Also, ADCs convert the final output to the digital form. This method is followed in several architectures such as ISAAC [37], iCELIA [10, 30, 44, 51].

Although these architectures allow for high-precision networks to be trained and inferred, the high cost of this approach appears in the power consumption and area. For example, the work presented at ISAAC [37] shows that the 8-bit ADC accounted for 58% of total power and 31% of the area. Also, it can be observed in iCELIA [44] that the cost of ADCs and DACs accounts for almost 90% of its power consumption. To solve that, the work presented in [7] lowers the bit precision of the architecture which corresponds to minimizing the cost of these blocks. However, it limits the usage of such architecture to networks that can tolerate such low bit precision. For example, the presented results showed limited accuracy loss in LeNet-5 but in cases of larger networks such as ResNet-18, it reaches 8% accuracy loss.

In-memory binary architectures, with the aforementioned problems in analog-based architectures and due to the limited number of states NVM can store; several studies regarding the usage of crossbar in-memory architecture for binarized DNN inference have been presented in [3, 8, 40, 48]. Though, these architectures show a high area and power efficiency with very high device reliability, their usage is still limited to a certain set of networks that can adopt such binary data representation.

In-memory digital-based architectures In [20, 29, 50] full precision full digital inference architectures were proposed. However, they lost one of the main advantages the in-memory architectures are offering by ignoring the advantages of the analog domain-based operations by activating only a single memory cell at a time. Also, in [29], this system showed very poor utilization in shallow layers. For example, if the system contains a large number of processing crossbars that can serve and benefit very deep layers, it performs poorly in the shallow layers compared to its peak performance. Also, the proposed operating frequency suggested by the work (2 GHz) leads to a high power consumption.

3 PRELIMINARIES

3.1 Convolutional Neural Network

CNNs are DNNs that target a wide range of applications including but not limited to computer vision-based applications. These networks typically consist of convolutional layers as the main component. In addition to convolutional layers, they usually have pooling and fully connected layers. As shown in Figure 1, a convolutional layer consists of a collection of kernels or weights that input feature maps are applied to and yield the output feature map. The (x, y, z) element of the output feature map can be expressed as follows:

$$f_o(x, y, z) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \sum_{h=0}^{C_{in}} f_i(x+i, y+j, h) K_z(i, j, h), \quad (1)$$

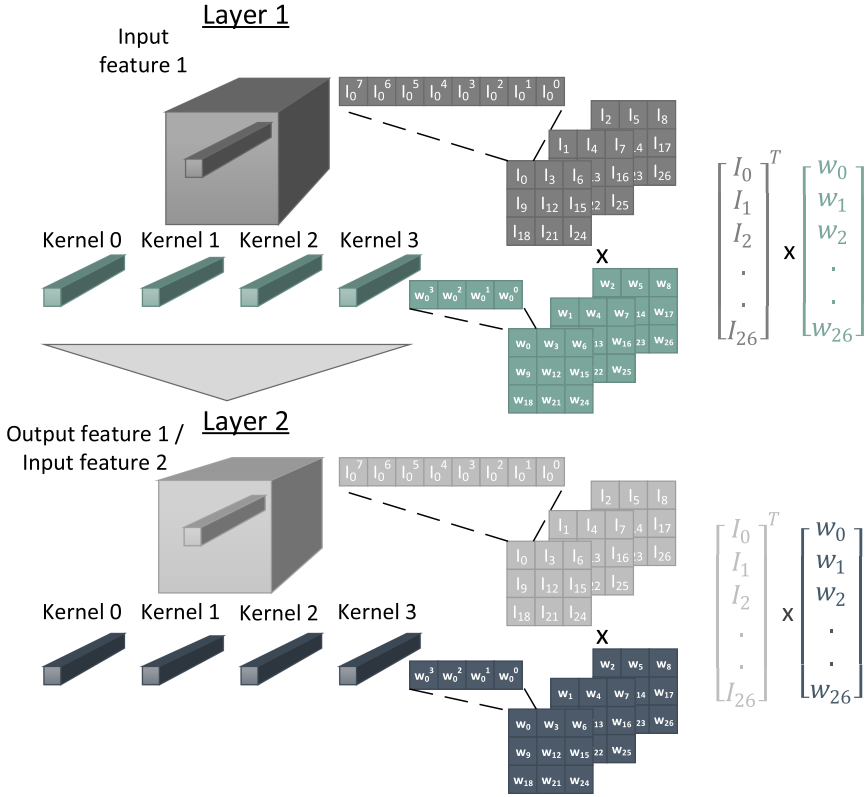


Fig. 1. Convolutional operation of two different layers showing a 1-D vector multiplication of the kernel weights and the convolution window of the input feature at each layer.

where f_o and f_i represent the output and the input feature maps, respectively. K_z represents the kernel with z as output depth. k and C_{in} represent kernel size and input depth respectively. The described operation is known as **Multiply and Accumulate (MAC)** operation which constitutes the major portion of the computational load.

Based on Equation (1), an arithmetic decomposition can be performed on the MAC operation to break it down into bit-level logic operations as shown in Equation (2), where I_p and W_p refer to input and weight bit precision, respectively. Also, w_s and i_s are used to ensure the correctness of 2s complement numbers format.

$$f_o(x, y, z) = \sum_{m=0}^{I_p-1} i_s * \left(\sum_{n=0}^{W_p-1} w_s * \left(\sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \sum_{h=0}^{C_{in}} f_{im}(x+i, y+j, h) \cdot K_{zn}(i, j, h) \right) \ll (m+n) \right). \quad (2)$$

$$w_s = \begin{cases} +1 & n < W_p - 1 \\ -1 & n = W_p - 1 \end{cases} \quad i_s = \begin{cases} +1 & m < I_p - 1 \\ -1 & m = I_p - 1 \end{cases}$$

As shown in Equation (2), the lowest level of operation is a logical AND between two bits to yield the final expected value. The summation of each set of operations needs to be shifted by $m+n$ to yield the expected output. Since the two operands are signed integers, in case of most significant bit in both weights and inputs, the result of the AND operations and shifting should be

Table 1. Example Bit-decomposed MAC Operation for $1 \times 1 \times 3$ Kernel, $W_p = 3$ and $I_p = 3$

$W = \{1(b001), 0(b000), -1(b111)\}$, $I = \{2(b010), 1(b001)\}$, $O = -1$				
	$n = 0$	$n = 1$	$n = 2$	Partial Sum
$m = 0$	$(1)^*(1)^*((1)\ll 0) = 1$	$(1)^*(1)^*((2)\ll 1) = 4$	$(1)^*(-1)^*((0)\ll 2) = 0$	5
$m = 1$	$(1)^*(1)^*((1)\ll 1) = 2$	$(1)^*(1)^*((1)\ll 2) = 4$	$(1)^*(-1)^*((0)\ll 3) = 0$	6
$m = 2$	$(-1)^*(1)^*((1)\ll 2) = -4$	$(-1)^*(1)^*((1)\ll 3) = -8$	$(-1)^*(-1)^*((0)\ll 4) = 0$	-12
Final Result = -1				

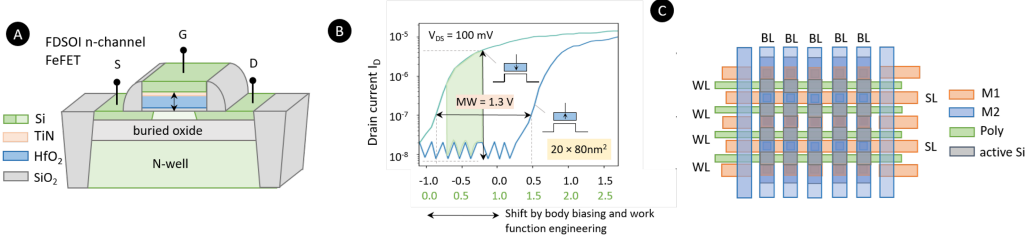


Fig. 2. (A) Schematic of a FDSOI n-FET device, (B) Transconductance curves as extracted from [14], and (C) Layout of the used 1-FeFET configuration.

subtracted not added. In Table 1, we demonstrate an example of a bit-decomposed MAC operation where the inputs and kernels are both of 3-bit precision. This decomposition allows for a high level of flexibility in terms of operands precisions as well as reducing the hardware footprint. Several architectures used similar decomposition approaches [4, 18, 27, 29] whether by serializing the input features or partially decomposing the weights. It can be observed that the usage of this technique offers various tradeoffs between throughput and hardware resources. It also allows for new levels of parallelization that can be used in addition to those already offered by CNNs.

3.2 FeFET Technology, Crossbar Array, and Interfaces

The FeFET is a well-known semiconductor device concept that until recently remained an unviable technology [31], despite the fact being patented as early as in the 50s and 70s. A FeFET contains a ferroelectric layer in the gate dielectric stack of a standard MOSFET. Recently, with the discovery of ferroelectric HfO_2 thin films the concept of FeFET was revived. The high coercive field (E_c), low dielectric constant, and CMOS compatibility make it very suitable compared to conventional ferroelectrics as lead zirconate titanate. Furthermore, its ferroelectricity is persistent down to ultra-thin films in the nanometer range [6], enabling highly scaled devices [5].

This enabled implementations into the 28 nm bulk [33] and 22 nm FDSOI [14] technology node. In FeFETs, the polarization state of the ferroelectric layer affects the transfer characteristics of the transistor, thus resulting in a shift of the threshold voltage V_t (Figure 2(B), as extracted from [14]). Due to the high coercive voltages and high remnant polarization values of HfO_2 [33], large **memory windows (MW)** are achievable, which are linked to a high on/off ratio [32] reaching values in the range of $10^3 - 10^5$. The low trap density and the low dielectric constant result in a low gate leakage current and a low operation voltage. The device-to-device variation of FeFETs is hereby mainly governed by the underlying polycrystalline grain dynamics [24].

The proposed architecture utilizes the implementation of FeFETs as demonstrated in the 22 nm FDSOI platform of Globalfoundries (22FDX). We illustrate the transconductance curve of an exemplary device given in Figure 2(B) (as extracted from [14]), with a gate size of $20 \times 80 \text{ nm}^2$.

For further analysis, we assume a V_t shift with respect to Figure 2(B) of 1 V. The assumed scale is highlighted in green. Such an offset V_t shift is obtainable by work function engineering. The offset V_t shift is hereby necessary to neglect any unintended leakage current within the crossbar array. Besides n-channel FeFET cells, also p-channel FeFET cells have been demonstrated, which make it possible to supply a complete non-volatile CMOS [26]. The transconductance of the FeFET is well behaved, which means that the temperature dependence simply results in a linear shift the V_t of the applied FeFETs [1]. The radiation hardness of the cells has been demonstrated for γ and heavy-ion irradiation [49].

Several implementations for **analog compute-in-memory (ACiM)** have been presented in the literature, utilizing various bit-cell concepts, often making use of access transistors. In this work, we are using a 1-FeFET bit-cell concept. The FeFET cell concept can shrink the bit-cell area down to the logic transistor size of the 22FDX technology of about $0.007 \mu\text{m}^2$. This contains **contact poly pitch (CPP) $\times$ minimum metal pitch (MMP)** as schematically illustrated in Figure 2(C). The general programming scheme hereby follows those previously documented concept for the FeFET [5], which involves the application of inhibiting voltages to prevent unintentional programming.

In inference mode, the activated FeFETs are selected via the **drainline (DL)** and the **word line (WL)**. The necessary **wordline (WL)** voltage to activate the 22 nm FDSOI FeFETs is assumed to be in the range of 0.4–0.8 V, mildly enough to prevent any disturbance of the cell. The non-activated WLs are inhibited to contribute by the application of a WL voltage of -0.6V . Note, that the voltages given here can change upon body bias and work function engineering. Due to the activation via WLs, the individual columns are only weakly capacitively coupled.

For such an aggressively scaled device with a W/L configuration of $20 \times 80\text{nm}^2$, a variation of σ_{V_t}/MW of 3 is expected in the current stage of technology maturity [5]. However, the operation in saturation mode as well as the implementation the 1FeFET1R configuration, reduces the influence of the device-to-device variation significantly [38]. In our configuration, we accumulate the results of eight activated FeFETs. The output current values for all permutations of input and weight configuration were evaluated. The individual states were hereby clearly separated without overlap.

4 FEFET CROSSBAR ACCELERATOR

In this section, we present our FeFET-based in-memory accelerator which is dedicated to DNNs. We start with a detailed discussion of each block across the various levels within the accelerator followed by an overview of our architecture. Throughout this section, we selected the different system parameters based on the design space exploration we performed. We start by defining the unrolling factors such as kernels unrolling factor, Kernel size, input size, and so on, at each layer within various networks. Second, we define for each system component parameter the area, power and throughput relation whether within the architecture itself in terms of impact on different subsequent blocks and busses or externally in terms of data movement and loads. From these information, we build a complete cost function that tries to optimize the system parameters to match the highest level of utilization across various layers within various networks while preserving certain conditions of total available FeFET capacity (8 MB). The final cost function has been built to adapt designer preference to optimize certain aspects over others. However, our chosen parameters do not emphasize a certain aspect over another and regard power, area and throughput equally.

4.1 FeFET Processing Element Design

The main building block of our architecture is the **Processing Element (PE)** where the analog operations are performed as well as the digital conversions. As shown in Figure 5, a PE consists of

the analog FeFET crossbar, mixed-signal blocks (**ADC**), and digital blocks (adders and subtractors). We explore each block structure and operation as well as their role in the architecture pipeline.

4.1.1 Crossbar Organization and Operation. As already explained in Section (2), a convolution operation can be decomposed into a set of AND operations through the input feature map and the weights. Similarly, each cell in the crossbar can perform a one-bit AND operation as explained. The crossbar is composed such that each unit cell stores a single weight bit. The gate, drain, and source of the transistors are connected to a WL, DL, and **source line (SL)**, as in Figure 3. The input feature map is fed into the WL and acts as row activation. The **source line (SL)** represents a control signal that specifies activated columns for each clock cycle. Finally, the DL yields the final result of the operation upon accumulation along various memory cells. The crossbar is divided into segments of 16×16 FeFETs, meaning 16 WLs $\times$ 16 DLs per segment, which improves the hardware utilization of the periphery circuits, such as the ADC. The ADC is connected to each column of 16 segments. The segments within the column share their DLs and since there are 16 segment columns per crossbar, we implemented 16 ADCs.

Every FeFET within such a segment can be programmed individually as schematically drawn in Figure 3. In order to program the FeFET in one of the two binary states, high V_t (HVT) or low V_t (LVT), a short pulse $V_{PROG} = 0$ V is applied and all the inhibit switches are set to 1 in order to discharge all the SLs from each the segment and the DLs from the crossbar to the ground. Afterward a WL voltage of $V_{prog, WL} = 3$ V is applied while floating the SL/DL. To prevent the other FeFETs within the same row (same WL) from being programmed, the rest of the inhibit switches are set to 0 and an inhibit voltage $V_{INHIBIT, SL/DL}$ via the V_{PROG} wire is applied at the SLs on the segment level and the DLs on the crossbar level (Figure 3). An additional inhibit voltage is applied to all WLs to prevent V_t state disturb of the FeFETs within the same SL and DL, which should not be programmed. This voltage is set to be 1 V. The preferred program pulse width here is expected to be 100 μ s. For erase, we assume a block erase with $V_{erase, WL} = -2$ V. Upon inference, the DL_SEL switch of the selected FeFET activates its SL within the segment and the DL within the crossbar and connects the FeFET to the ADC.

Besides the FeFETs, we use a resistive element—forming a 1FeFET1R bit-cell—to limit the current of the FeFET and reduce the input current variation for the ADC. Therefore, we use a current generator to supply a reference current of about 100 nA, which is transferred into the FeFET column per segment via a current mirror with a similar latency as the ADC. The reference current for the ADC has been designed as 100 nA too. The strongly reduced current variation for the output “1” for bit-wise multiplication operation, makes it possible to clearly distinguish the accumulated number of activated FeFETs in the LVT state upon row activation. During the inference process, only one FeFET per segment is activated and it can contribute to the total current. Therefore, the SL of the activated FeFET is connected with the resistive element/current mirror (Figure 3) and its DL is connected with the ADC. The total current of the 8 segments is connected to the ADC which senses the result of the MAC operation.

4.1.2 Current-based ADC. Unlike common ADCs, our ADC operates in high side current mode with a thermometer coding as in Figure 3. It consists of a group of stacked current mirrors as the PMOS transistors and the NMOS which are responsible for forwarding the measurement current to the next stage of the ADC. Each PMOS in the current mirrors is responsible to create a reference current of 100nA.

As the input current is smaller than the reference current, the first current mirror maintains a high voltage potential at its output Out1. The Out1 is also the input of the ADC and it is connected with the segment column. If the input current rises beyond the reference current I_{ref} , the voltage at

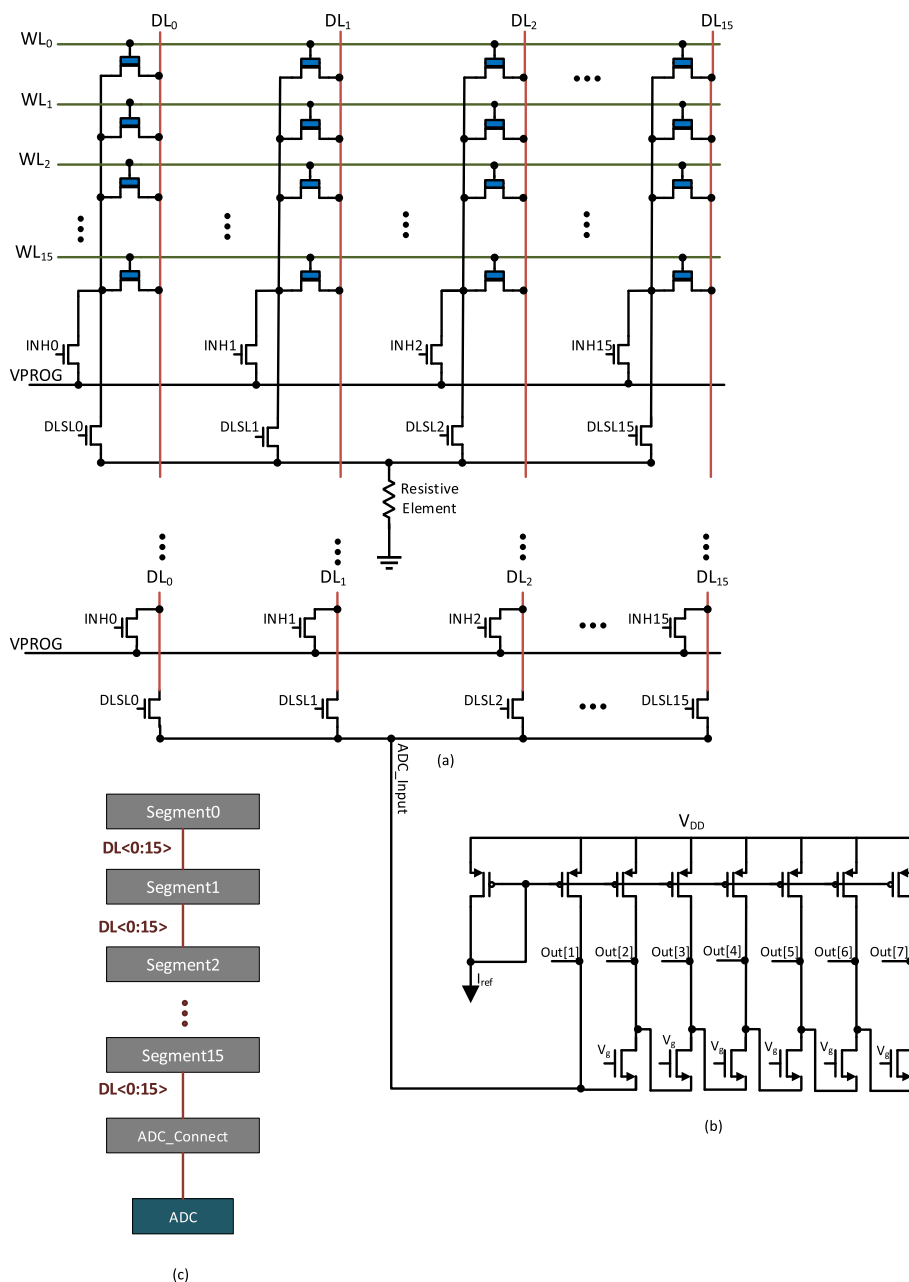
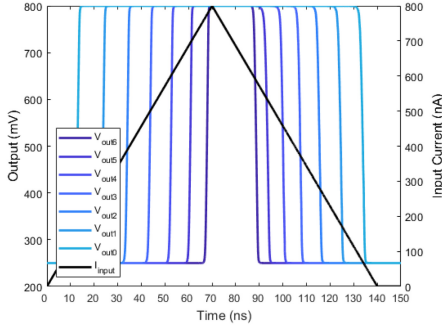
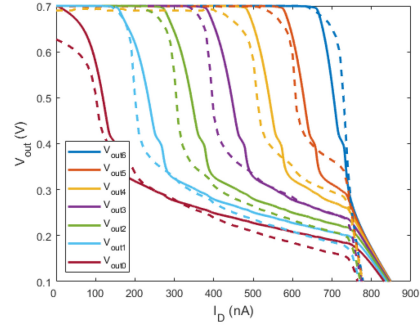


Fig. 3. (a) The circuit design of a segment with the resistive element to form 1FeFET1R bit-cell with the ADC_connect circuit to enable programming and inference mode, (b) the column of the 16 segments connected with the ADC, and (c) the 3-bit thermometer code ADC.

Out1 drops sharply. If the output potential at Out1 drops, the source-gate voltage of N6 increases. This turns on N6, which then provides a bypass for any extra current drawn by the input. If the input current keeps rising and exceeds the set current of the P1/P3 mirror, the voltage at Out2 will drop too. The process repeats for all subsequent stages of the ADC. Finally, the ADC is followed by



(a) Simulation of a seven stage staircase ADC with inverting buffers in 22nm FDSOI.



(b) Measurement (dashed line) and Simulation results (line) for the ADC outputs over the input current.

Fig. 4. Simulation and measurement results of the 3-bit thermometer code flash ADC.

an encoder responsible for buffering the ADC outputs and yielding the final binary result which is used afterwards.

The presented current-based ADC shows very high efficiency in terms of power and latency and it has only a small footprint and resolution. This fact does not limit the usage of this ADC in our architecture as we target low-precision ADCs. In Figure 4(a), the sequential activation of the ADC outputs is presented. For simulation purposes, the time step per activation is set to 10 ns. The simulation and measurement results of the ADC outputs with respect of the input current is presented in Figure 4(b). The simulation results are plotted with the line and the measurement results with the dashed line.

4.1.3 FeFET Crossbar Section. Each crossbar is split into sections. In each section, groups of FeFET segments are formed where each group is connected to a specific ADC. In this configuration as previously highlighted, only one FeFET is activated per segment at any time. Such organizations limit the number of ADCs within the crossbar to the columns of the FeFET segments instead of the columns of FeFETs. Also, it limits the resolution of the ADCs as only one FeFET is activated per connected segment to the ADC. Such an approach drastically reduces the area and power of such a crossbar. Furthermore, it allows for using the crossbar as a memory for various layers instead of using the entire crossbar for a single layer. However, such a limited number of possible activations does not affect the throughput compared to existing architectures as it allows for high frequency. For example, in ISAAC [37] though they activate the entire crossbar 128×128 they can run it only every 100 ns. This is equivalent to 163.84 Giga Activation per second. Also, activating the entire crossbar simultaneously drastically reduces utilization at networks not matching crossbar size. On the other hand, our architecture can activate the cell every 1 ns yielding up to 256 Giga activations per second while maintaining achievable parallelization levels as shown later which translates into a high level of utilization at different workloads.

In Figure 5, an example of the crossbar section is illustrated. Here, the convolution operation is unrolled by the following factors. The $\sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \sum_{k=1}^{C_{in}}$ is parallelized by the factor of the simultaneously activated FeFETs per group. Also, $\sum_{n=0}^{W_p-1}$ is parallelized by the factor of groups per crossbar section. For example, in case of 4-bit weight quantization, our architecture will consider only four groups per section.

The number of activated FeFETs per group also defines the required ADC precision. In our architecture, we activate 8 FeFETs at a time each from a different segment which corresponds to an 8 states ADC. However, approximating the output value to a 3-bit value (7 states) does not

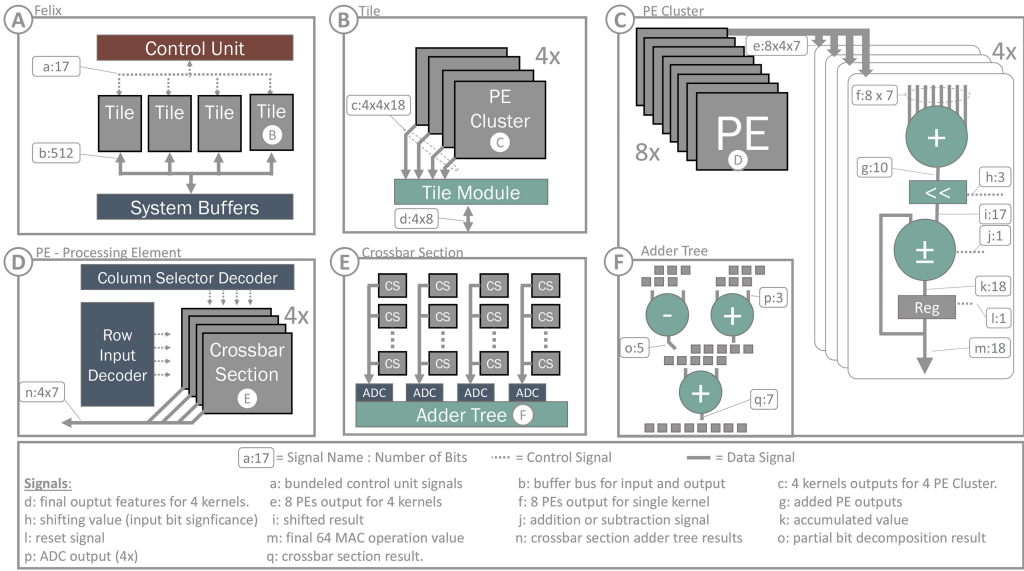


Fig. 5. The overall system architecture in addition to explanation and bit-width of each system signal in case of 8-bit activation and 4-bit weight. (A) Different systems tiles are connected to the control unit and buffers. (B) System tile consists of PE Clusters all of which are connected to tile module. (C) PE cluster consists of PEs whose results are added, shifted, and accumulated. (D) The PE is built from different crossbar sections in addition to column and row decoders to activate the correct FeFETs every clock cycle. (E) Each crossbar section consists of the presented **crossbar segments (CS)** such that each column of CSs is connected to an ADC. The results of the ADCs are connected to the presented adder tree to yield a partial result for further processing. (F) The presented adder tree shown here is used in the case of 4-bit weight accuracy.

affect the network's accuracy due to the bit-level sparsity. As illustrated, the 3-bit results out of the ADCs are followed by adders to form a partial result for the MAC operation. Each ADC result is shifted by a value of n mentioned in (2) which corresponds to the weight bit significance computed by the ADC. It can be observed, that one of the ADC results is subtracted to maintain the correctness of signed operations as this ADC computes the operations of the weights sign bit. The adder yields a partial sum that is routed to cluster adders (explained in the next section) for further processing.

4.1.4 Overall Processing Element Crossbar. Within each crossbar, we form four sections that operate in parallel. From a hardware perspective, the four sections do not have any physical separation. It is an algorithmic point of view where each section represents a different kernel. However, the inputs applied to the crossbar rows are applied to all of them. Since all four sections have the same input feature map, they can parallelize the computation of different kernels. Each section has its own set of ADCs and adders that should yield a partial sum for a different kernel output feature. In our architecture, we use crossbars of size 256×256 FeFETs which consists of 16×16 FeFET segments. Each 3-bit ADC is connected to 16 FeFET segments. For the presented system configuration, only 8 segments are activated.

This crossbar organization balances the arithmetic operations between the different blocks while lowering the data traffic needed. At any point in time, it only needs 8 bits of input (in case of 8 activations) and only four result words each of seven bits needs to be routed to the next block for further processing.

In addition to the crossbar sections, a row/input decoder is used to activate the current computed weight row in case of a value of 1 is applied as input. Also, a column selection module is used to select the correct column.

4.2 PE Cluster

As each PE computes a single input feature map bit significance at a time, the PE cluster collects the results from the enclosed PEs and further accumulates the values over time until all bits of the input feature map have been computed. As illustrated in Figure 5, each cluster consists of eight PEs that compute a single-bit operation for each of the 64 input feature maps across four different kernels. The results of different PEs are then added together. To do that, each cluster has four adders (one for each kernel) with eight inputs each. The results of these adders are then shifted by m bits to the left which represents the significance of the currently computed input feature map bit. The shifting value signal is generated by the central control unit.

Finally, these values are accumulated through four different accumulators to form the final result. In the case of the most significant bit of the input feature map, it is subtracted instead of accumulated to yield a correct signed MAC operation result as this bit represents the input sign-bit. The iterations needed for accumulation depend solely on the input feature map precision. Through this pipeline, the PE cluster allows for further reducing the data traffic by only transmitting four words, 18-bit each in case of computing 64×8 -bit activation/4-bit weight MAC operations.

4.3 System Tile

Our system consists of tiles of the presented clusters in addition to a special tile module. As DNNs are known for the high number of MAC operations reaching thousands of operations for each output feature, the presented cluster can perform only 64 MAC operations which can increase the latency per single output feature.

Therefore, to minimize the single output feature latency, we form tiles of clusters that can further parallelize MAC operations. The results of the different clusters within the tile are received by the tile module as shown in Figure 6. Depending on the number of accumulations per output feature at the computed layer, either the results of two or four clusters are added or even accumulating the results in case of MAC operation larger than the four clusters capacity. Hence, in the case of only 64 accumulations needed per output feature, the tile module yields 16 final output features (Four features for each kernel). Also, if the accumulations needed were more than 256, then the accumulations continue until the final value is ready. In the case of pooling following the computed layer, the yielded features are stored and compared or averaged with the next output features and only the pooling result is stored. The tile module also has an activation unit for applying any activation function to the output feature (ReLU, tanh, sigmoid, etc.) and performing the final re-quantization to 8-bit.

The multiplexer responsible for selecting the output feature, the accumulator, activation unit and the pooling unit receive control signals from the central control unit.

4.4 Top Level Architecture

The final system is composed of several tiles. Though each PE already performs the computations of four kernels, this parallelization has to be further increased as convolution layers usually consist of a large number of kernels. We further parallelize computed kernels across the different tiles. If r is the number of system tiles, then the overall system parallelizes the kernels that can be computed by the factor $4r$. If the output feature map depth is less than or equal to $4r$, then the full output feature map depth will be computed simultaneously.

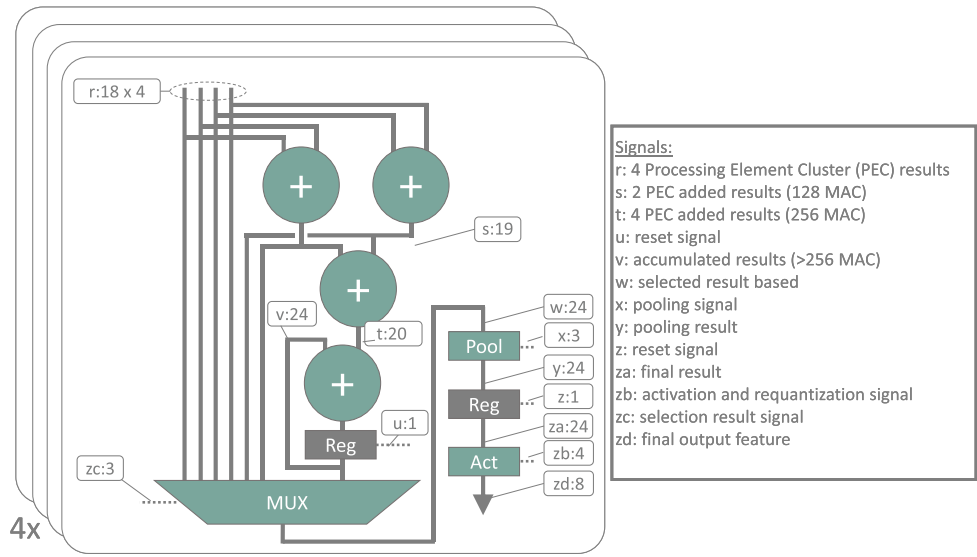


Fig. 6. The tile module can further add and accumulate the results of the different clusters as well as perform pooling and activation operations. The tile module operates on four kernels which yield four final output features.

Such structuring allows for maximizing the data reuse. As different tiles are computing different kernels, they can receive the same input feature map. This high input data reuse allows for reducing the data to be loaded from buffers as well as the complexity of the loading mechanism and finally the overall data traffic across the whole system. In Figure 5, we illustrate the top-level design of the system including a central control unit and buffers.

4.4.1 Buffers. The tiles grid is complemented with a buffer unit that is split into two main partitions; an input feature map buffer and an output feature map buffer. The input buffer is further split into partitions such that each partition is storing the input feature bits targeting a certain cluster in each tile. This partitioning and the later explained mapping are done to preserve a one-to-one relation between the buffers partition and the row of tiles which simplifies the reading process from the buffers. The mapping of the partition content is defined prior to inference through a framework that we built. During buffering of the input feature map, words that are built from bits from different features are stored. This eases the loading process from the buffers to the different PEs. The second partition of the buffers is the output feature map partition. As it is preceded by the pooling unit within the cluster column, only the result of this unit in the case of pooling layers is stored. This partition is split into two different partitions to implement a double buffering scheme. The first one receives and stores the currently computed features while the other partition stores the features from the previous iteration in the external memory. Each cycle the partitions swap their roles. The size of the partitions is determined based on the performance analysis to make sure that at any point the system will not be limited by input/output feature memory load. Such a partitioning scheme requires an external memory burst rate (memory bandwidth) of 5.3 GB/second at 100% utilization of the system.

4.4.2 Control Unit. The central control unit is responsible for initiating the pipeline and generating the different control signals mentioned for different system blocks such as shifting and resetting signals. At the start of each layer, the control unit receives different layer parameters

that include the input feature map precision. This precision defines the number of iterations the crossbar needs to complete the operation. Also, it receives the input feature map depth and kernel size; which define the required accumulations and the results of the **PE** clusters that need to be added. Also, the control unit receives the kernels' mappings in the crossbars to activate the corresponding columns and map inputs to the correct rows. Such mappings are done statically and the weights are stored in the crossbar sub-arrays according to the mapping. Finally, it receives the number of kernels which defines if the input needs to be used more than once on the grid to compute different kernels if the number of kernels was more than the kernel parallelization factor.

5 MAPPING AND ACCELERATOR OPERATION

In this section, we describe how the weights are distributed over the crossbar cells to maximize the system utilization as well as fulfilling the mapping requirements of the segment and pipeline operation of the system.

5.1 Kernel Weights Mapping to the Crossbar Array

As discussed in Section 4, the 256×256 crossbar is organized as follows: it is divided into four sections each consisting of 64 columns and each of these sections is further divided into four vertical groups (16 CS each) where each segment contains a 16×16 FeFET. Every 16 segments share an ADC. The four sections of the crossbar represent four operating kernels in parallel, fed with the same input features (four kernels of the same convolutional layer operating at the same time). Each column of segments in the crossbar section represents one bit-significance of the quantized kernel weights. So during each layer calculation, in case of 4-bit weight quantization, four columns (one from each group) in each region (kernel) are activated. Additionally, in order to ease the requirements of the size of the ADC as previously explained, eight inputs (rows) are activated during each cycle in each PE. An additional restriction is that only one bit-cell in each 16×16 segment of the crossbar is activated.

In Figure 7, we showed an assumed explanatory structure of two convolutional layers each consisting of two kernels. The two kernels of each layer need to be mapped to two regions in one PE to operate in parallel. Each kernel is linearized as a 1-D column vector. If the kernel weights are quantized to 4-bits, the 1-D column vector can be decomposed into four column vectors each storing kernel weight bits of certain significance (bit0, bit1,...) which then is mapped to one column in the crossbar array. Those columns are in the same region, and each column is in a different group to operate simultaneously as shown in Figure 7. The 1st kernel of layer1 is mapped to columns 0, 16, 32, and 48, while the 2nd kernel of layer1 is mapped to columns 64, 80, 96, and 112. Concerning the occupied rows, as mentioned only one bit-cell should be activated from each segment, so the mapped kernels can occupy rows 0, 16, 32, and so on, such that the linearized 1-D vector is mapped as groups of eight weights in that manner.

The two kernels of layer 2 can be mapped in the same manner to different columns sharing the same groups and regions. During the calculation of each layer, the controller unit has to activate the corresponding bit-cells.

5.2 Accelerator Operation Timeline

As mentioned, the accelerator operation bit-decomposes both the input feature and the kernel weight. The bit decomposition for the weights is achieved by their mapping to the crossbar array bit-cells. The input features are bit decomposed and applied in a serial way to the PEs crossbar each cycle as shown in Figure 8. The number of clock cycles the operation needs will depend on the input precision. In Figure 8, we illustrate the timeline for the accelerator operation. During the first cycles, the input is fed serially and the results of PE units are added together by PE cluster

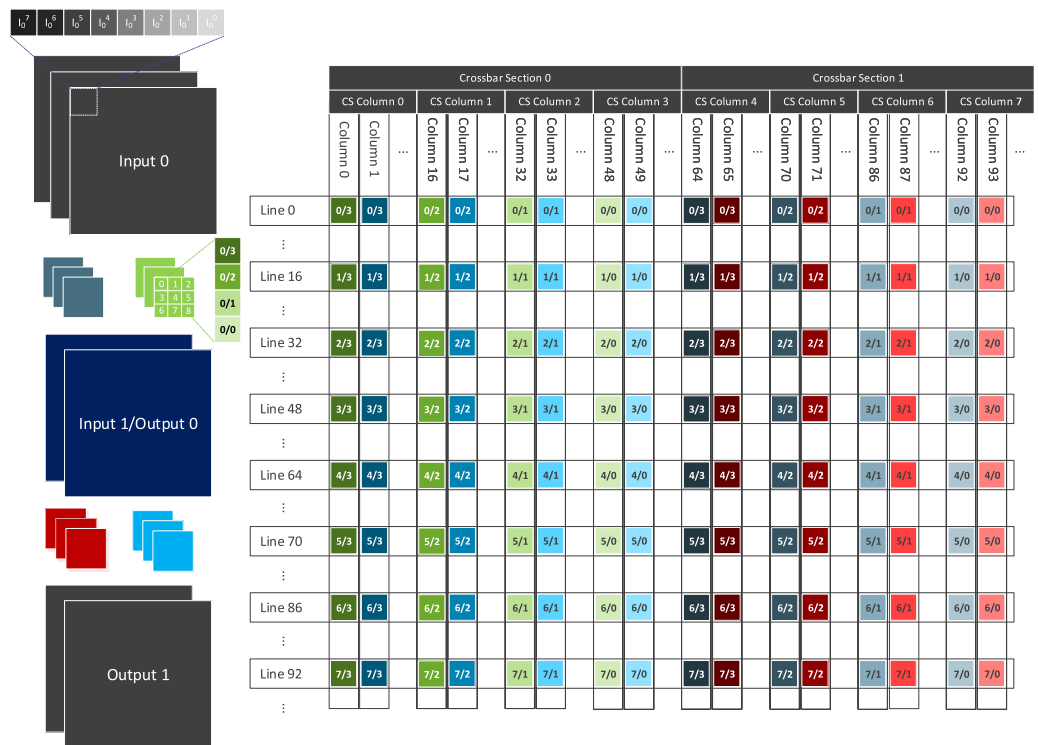


Fig. 7. The memory mapping of two kernels from two consecutive layers into the crossbar array. Only eight weights from each kernel are shown, however, the rest of the kernel is mapped similarly such that every eight weights are activated simultaneously. Each crossbar section contains four CS and each consequent 16 lines map to rows of a crossbar segment.

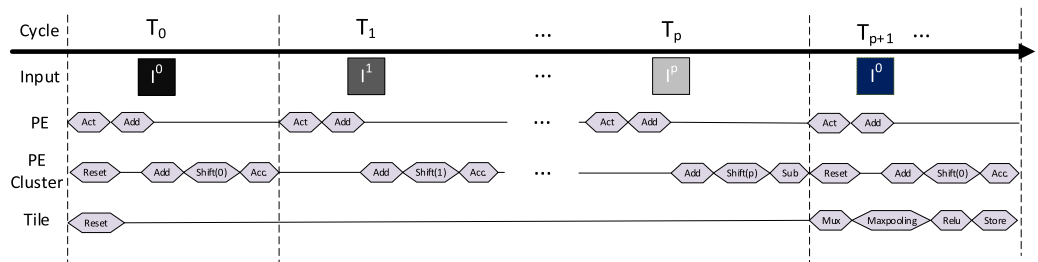


Fig. 8. The architecture pipeline where the computed input feature bit significance defines the operation performed across the hierarchy.

addition trees. Then, they are shifted and accumulated each cycle in the PE clusters' accumulators. After serially feeding all input bits, the tile module selects the correct output depending on whether it is the addition of several tiles results or a single one. Afterward, the pooling operation can be performed if needed as well as the activation function. Finally, the output feature is stored in the output feature map buffer. An accumulation step in the tile module is needed if the kernel size is larger than 256 and the output feature cannot be calculated in one calculation step.

In order to increase the system throughput in case of low computational load layers, the kernel weights can be duplicated into other PEs. This allows for two further parallelization dimensions.

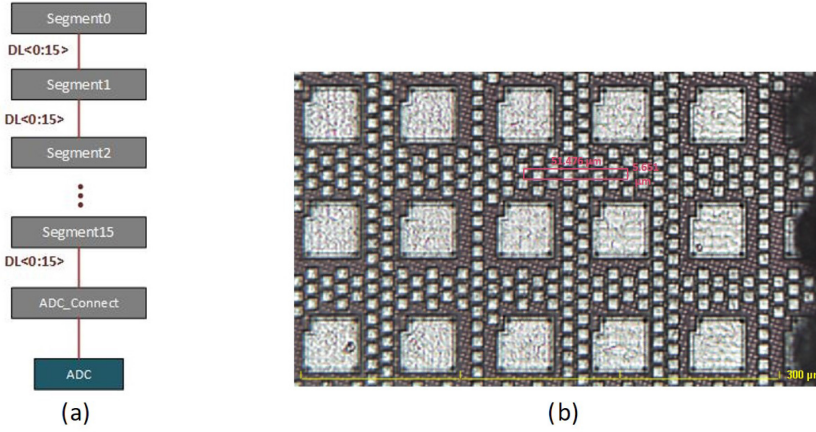


Fig. 9. (a) The design of the column segments with the ADC and (b) the taped-out wafer of the design.

First, the input features bits are distributed among them. For example, the most significant half is fed to one PE, and the least significant half is fed to another PE in a different PE cluster. As a result, this would cut the latency of single-feature computation time in half. The second possibility is to compute several output features at different clusters which will double the throughput or even more depending on the number of replications in shallow layers.

6 EVALUATION

In this section, we first show the experimental setup and the different benchmarks used including their accuracies. Then, we evaluate our architecture and explore the experimental results from several aspects such as accuracy, utilization, power, area, and speed.

6.1 Experimental Setup

We used Cadence Virtuoso to evaluate the mixed-signal blocks where the transient behavior of the ADCs, the associated column, and the current limiter were investigated. Process variation of the circuit elements was investigated by Monte-Carlo simulation at room temperature. The MAC accuracy of the output at the thermometer-code ADC is analyzed and compared to the expected results. For suitably small σ_{V_t} of the FeFET and the chosen 22FDX technology, we obtain fault-free MAC operation computation. Also, we use Synopsys Design Compiler tools to evaluate the different digital blocks. Finally, we estimate the full system area and power consumption by combining all the circuit simulations and measurements and determine the overall overhead and latency. We built our simulation framework using ProxSim [11] which is based on Tensorflow. We used it to evaluate the CNN accuracies considering our performed bit-decomposition as well as the variability introduced by the FeFET cells and the ADCs.

In Figure 9, the taped-out wafer of the column segments with the ADC is presented. The total area of the crossbar column is $51.476 \times 5.651 \mu\text{m}^2$ which makes it hard to be distinguished on the pads-scaled wafer. The measurement results with respect to the simulations are given in Figure 4(b). As can be seen, the measurement results match the simulations except for the first output (input) of the ADC where there is a small leakage which results in a lower voltage level.

In Table 2, we summarize the different parameters of our architecture as well as the operating frequency. As we only consider 1,024 PEs within our design, the maximum size of the network the architecture can support is 64 Mb. Though the architecture can adapt to different weight precisions, we map and compute different convolutional models at the accuracy of 8-bit/4-bit (input/weight)

Table 2. Design Parameters for the System Prototype

FELIX System at 500 MHz				
Component	Params	Value	Power [mW]	Area [mm ²]
CS	Size num/PE	256 b 256	0.017 uW	17.42 um ²
ADC	Resolution Freq. num/PE	3-bit 1 GSps 16	0.31 uW	8.54 um ²
1 PE Total			0.02468	0.0046
1 Cluster Total	num. PE/Cluster	8	0.341	0.037
Tile Module			0.089	0.000289
1 Tile Total	num. Cluster/Tile	4	1.59	0.148
Buffers	Size	1 MB	5.12	0.131
Control unit			0.082	0.000389
Chip total	num. Tile/Chip	32	56.1	4.9

as it presents a good balance between the benchmarks’ accuracy and system performance. We use a batch size of 1 for the selected benchmarks to represent worst case of parallelization from an input perspective as our architecture targets low power inference applications. We benchmark our system with Squeezenet [16] for Imagenet [12], Resnet18 [15] for Imagenet, Resnet20 [15] for CIFAR10 [21], MobilenetV2 [35] for CIFAR10, Dialated model [45] for Cityscapes [9], and LeNet-5 [23] for MNIST [13]. These benchmarks represent different computational and memory loads as shown in Table 4.

Finally, we built a mapping framework based on Section 5 which is responsible for defining the storage unit for weights of different benchmarks. Also, we built a simulation framework based on the different system estimations to measure system utilization and performance across different benchmarks.

6.2 Benchmarks Accuracy

Since our architecture performance depends on the precision of the inferred networks, we tested the applicability of the usage of 8-bit/4-bit quantization (input/weight) based on the optimizations presented in [17]. We measured the classification accuracy over the test dataset in the case of (CIFAR10 and MNIST), and over the validation dataset when using ImageNet. For the dilated model, the Mean Intersection Over Union is reported as the CNN accuracy over the validation dataset.

As shown in Table 3, compared to the floating-point (Top-1) accuracy the 8-bit/8-bit maintains the accuracy with a maximum 0.6% accuracy loss. In the case of 8-bit/4-bit, we were still able to maintain accuracy with a maximum loss of 2%. Despite using highly optimized quantization (8-bit/4-bit), the system FeFET variations, and the ADC leakage, our architecture maintains networks’ accuracy with almost no accuracy loss (<0.2%). In the following section, it can be observed how such quantization optimization can boost the system performance and use the advantage of flexible precision the system is offering.

6.3 Performance Analysis

In this section, we benchmark our architecture across the different presented networks. We focus also on how our system can achieve high average utilization across different computational loads to maximize the performance in comparison to the peak performance.

Table 3. The Benchmarks Original Accuracy at Floating-Point Representation Compared to the Different Presented Quantizations as Well as the Effect of the Various Variations on the Architecture Accuracy

Network	FP Top-1 Acc.	8/8-bit Acc.	8/4-bit Acc.	FELIX Acc.
LeNet-5	99.11%	99.07%	98.86%	98.86%
Resnet20	91.04%	91.04%	90.6%	90.6%
MobilenetV2	94.89%	93.7%	93.7%	93.7%
Resnet18	68.68%	68.08%	68.06%	67.98%
Squeezenet	56.67%	56.19%	54.2%	54.02%
Dialated model	63.08%	62.85%	62.43%	62.38%

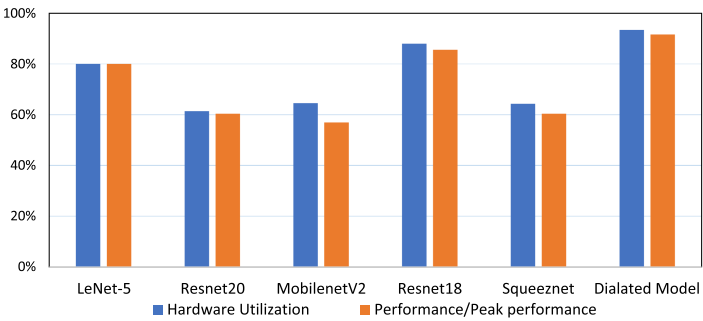


Fig. 10. Hardware utilization across the different benchmarks and the corresponding performance compared to peak performance.

For the computational performance, our architecture can reach a peak performance of **2.05 tera operations per second (TOPS)** where each operation is a full MAC operation for 8-bit input and 4-bit weight.

As shown in Figure 10, our system shows a direct relation between the system hardware utilization and the achieved performance compared to the peak performance without any effect related to the model size. This means that our architecture is achieving a complete independence from the external memory latency. Such efficiency was achieved through the locality of weights throughout the inference process. Also, the effective double buffering for inputs and outputs which masks the memory-related transfers. Additionally, our system utilization depends on the computational load of network layers as shown in Table 4. It can be observed that as the computational load per output feature increase the corresponding utilization as well. This yields a high overall network utilization as the layers with high computational loads represent the major bottleneck. However, the utilization can be affected by the dimensions of the layer compared to the parallelization factor. Also, as explained previously we integrate the pooling layers with their previous layer so that any extra overhead related to their tasks can be avoided. Overall, the system shows high flexibility which still allows it to reach a high utilization across different loads reaching 93% as shown in Figure 10. Also, as shown in Table 5, our architecture maintains a high throughput and energy performance across various benchmarks. The system also shows a good extendability to larger dimensions of PEs and crossbar sizes while maintaining a high computation efficiency. However, in order to keep high utilization in such extended dimensions, replications of the weights need to be stored.

6.4 Energy Efficiency

With energy efficiency ranging from 1,169 TOPS/W (1-bit/1-bit) to 18.27 TOPS/W (8-bit/8-bit), our architecture shows a huge edge in terms of energy efficiency as shown in Figure 12. Such

Table 4. Resnet-18 Topology and the Utilization of the System Across each Layer

Layer	Output Size	Kernel Size	#MAC/feature	#MACs	Utilization	Latency (#cycles)
Conv1	$112 \times 112 \times 64$	stride 2, 7×7 , 64	147	118 M	57.4%	50,176
Maxpool	$56 \times 56 \times 64$	stride 2, 3×3	-	-	Integrated	-
conv2_x	$56 \times 56 \times 64$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	576	462.4 M	75%	150,528
conv3_x	$28 \times 28 \times 128$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	1,152	368.6 M	86.25%	112,896
conv4_x	$14 \times 14 \times 256$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	2,304	368.6 M	97.5%	100,352
conv5_x	$7 \times 7 \times 512$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	4,608	368.6 M	100%	98,784
Average pool	$1 \times 1 \times 512$	average pool, 7×7	-	-	Integrated	-
FC	1,000	$512 \times 1,000$	512	0.5M	97.6%	142

The x in convN_x refers to the consequent layers within the model of the same structure.

Table 5. Benchmarking Models

Network	#Input [M]	#Param [M]	#MAC [GOPs]	Perf. [FPS]	Throughput [TOPS]	Energy Perf. [TOPS/W]
LeNet-5	0.005	0.062	0.00039	4.6 M	1.64	32.15
Resnet20	0.211	0.271	0.039	30.2 K	1.25	27.23
MobilenetV2	2.04	3	0.28	4.2 K	1.19	26.2
Resnet18	2.383	11.7	1.7	960.5	1.74	33.2
Squeezenet	1.743	0.722	0.27	4.3 K	1.24	27
Dialated model	34.96	4.957	49.44	37.9	1.87	34.7

performance can be accounted for the system’s very low operating voltage and current. Also, as shown in area efficiency, the innovative **ADCs** presented that are customized for our in-memory architecture and elimination of **DACs** account for a large reduction in power consumption as shown in Figure 11. We also drastically reduced the number of simultaneously operating cells within the crossbar which effectively makes the major power consumption comes from the digital blocks (adders, shifters, etc.). This allows for further power reduction as these blocks offer various effective approximation opportunities that can still preserve network accuracies while increasing energy efficiency.

Compared to the state-of-the-art architectures, our design shows a reduction of average power consumption down to 56.1 mW as shown in Table 2. This corresponds to energy efficiency of 36.5 TOPS/W (8-bit/4-bit). With such performance, our system outperforms the top-performing state-of-the-art in-memory architectures by a factor of 1.63X even on the macro to full system comparison.

6.5 Area Efficiency

We also analyzed the area efficiency of our architecture. In this area investigation, we included the crossbar and the **ADC** size. In Figure 14, the layout and the area of the crossbar with the ADC

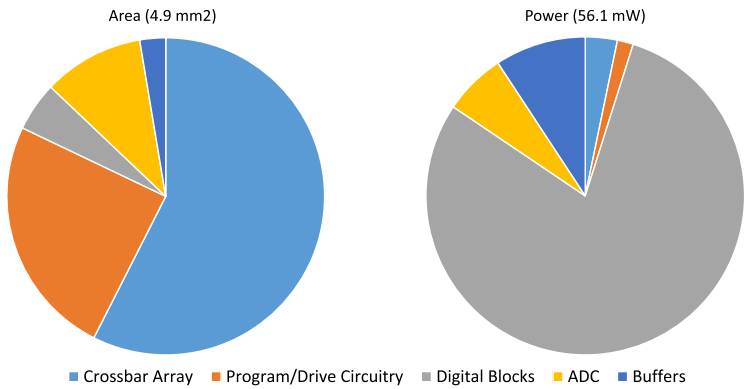


Fig. 11. System area and power breakdown.

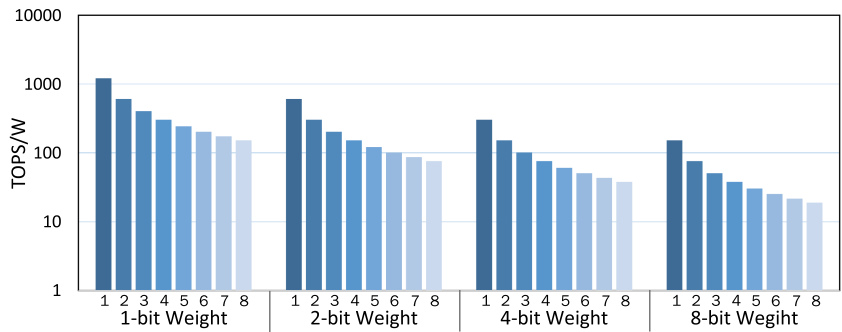


Fig. 12. System power efficiency in TOPS/W at different input and weight precision.

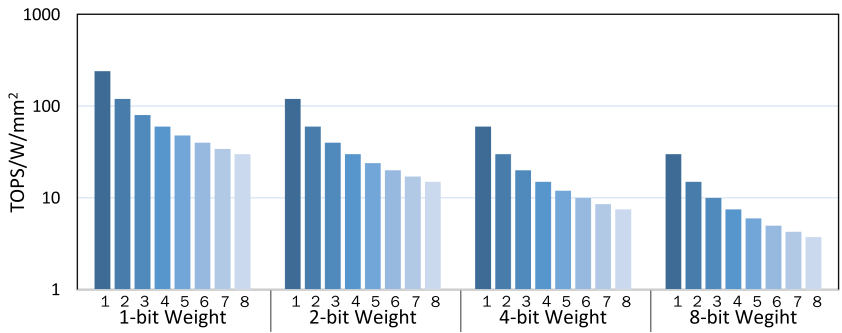


Fig. 13. System area efficiency in TOPS/W/mm² at different input and weight precision.

is illustrated. As can be seen, the total area of the crossbar with the ADC is around $85 \times 50 \mu\text{m}^2$ which makes it a very competitive design in terms of area efficiency. Also, we included the adders, accumulators across the PEs and the overall system, the needed input/output feature map buffers, and finally the control unit logic needed for the system operation. A detailed breakdown of the system is shown in Table 2. The design occupies an area of 4.9 mm^2 .

As our system can allow for completely flexible input and weight precision with few changes in ADC output connections. In Figure 13, we show the system performance at different precisions.

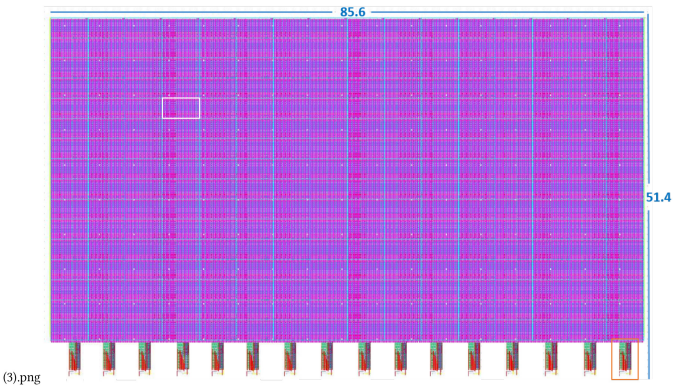


Fig. 14. Layout of the crossbar configuration with the ADC. The white frame represents the segment and the orange one the ADC.

Table 6. Comparison between FELIX and Ferroelectric Based In-Memory Architecture

	FELIX	VMM based Arch. [29]
# of PEs	1,024	2,048
PE Area [mm ²]	0.0046	0.00315
System Memory Capacity [MB]	8	16
Frequency [GHz]	0.5	2
Peak Memory Bandwidth	5.3 GB/s	512 GB/s
Total Power [W]	0.056	18.2
Total Area ¹ [mm ²]	4.9	43.9

Ranging from 261.1 TOPS/W/mm² for binary operations to 4.08 TOPS/W/mm² for 8-bit/8-bit (inputs/weights), our system shows a very high area efficiency where the architecture can accommodate a network with a size up to 64 Mb.

As shown in Figure 11, such performance can be accounted to the huge reduction in the area needed by the small 3-bit ADCs and the usage of small FeFET cells. Furthermore, the optimized architecture data accumulation reduces the data to be transported which reflects on the buses area.

6.6 Comparison to DNN Accelerators

In Table 6, we compare our architecture and another FeFET-based architecture. It can be observed that we maintain a more feasible external memory bandwidth and frequency without compromising our architecture performance as shown in Table 7. Though we occupy a larger PE area, our PE design drastically reduces the number of consecutive adders, accumulators, and network on chip requirements which reduces the final system area. Also, Our PE includes the size efficient ADCs which allows for simultaneous activation of FeFETs while the compared architecture can only activate a single FeFET per crossbar column and consequently needs 256 counters and adders for each PE. Finally, we require much lower memory bandwidth due to very high data reuse and lower operational frequency compared to the other FeFET architecture.

Furthermore, in Table 7, we compared our architecture to further advanced accelerators including the pure CMOS-based UNPU and several in-memory architectures that vary in their computational paradigm. We scale the different architectures process into our process for a fair comparison. Despite our system adaptation of more flexible and balanced mixed-signal processing as shown in

Table 7. Simulation Results for Various In-Memory Computing Systems using Different Technologies (System-level comparison)

	PipeLayer [41]	ISAAC [37]	Lattice [50]	UNPU [27]	VMM-PIM [29]	FELIX
Technology	28 nm	28 nm	40 nm	65 nm	28 nm	22 nm
Parameter Storage	ReRAM	ReRAM	ReRAM	CMOS	FeFET	FeFET
Storage [bits/cell]	4	2	1	-	1	1
Power [W]	-	65.8	-	0.12	18.2	0.056
Area ¹ [mm ²]	73.2	75.69	-	9.3	43.9	4.9
Peak Perf. [TOPS]	-	-	-	1.38	16.38	2.05
Energy Perf. ² [TOPS/W]	1.68	2.9	22.28	11.6	0.896	36.5
Area Perf. ^{1,2} [TOPS/mm ²]	3.34	0.84	3.64	0.14	0.37	0.51

¹Technology scaling to 22 nm technology using DeepScale [36].

²Normalized to Int8/Int4 (Int8/Int8 op = 2 Int8/Int4 op).

the previous section, it outperforms still the presented architectures in energy efficiency. However, Pipelayer and ISAAC show higher area efficiency due to their ability to store more than a single bit in their used ReRAM cell. In case of Lattice, area performance is not completely comparable since they only consider the area of the PE macro and none of the other system components such as buffers, control unit, and so on. Also, in our comparison, we scale the entire ReRAM area to our process even-though the ReRAM cell does not necessarily scale as well. Also, our architecture is lower in terms of peak performance as we keep the number of PEs to a level that can guarantee a high level of utilization at all points of time which represents a major drawback for several presented works.

Also, as previously highlighted, the system supports various networks with different complexities at low quantization and high accuracy compared to the state-of-the-art. For example, in Pipelayer, the network accuracy sharply deteriorates at lower resolutions and needs to have a high quantization precision. Also, our architecture shows a better ability to integrate more approximations, whether through flexible quantization or at the various system blocks that can tolerate approximation without affecting the network accuracy. We could not include a full comparison on a specific network in terms of performance and accuracy as many of the state-of-the-art architectures refrain from presenting their performance on specific networks for various understandable reasons.

7 CONCLUSION

In this work, we presented an in-memory architecture based on an arithmetic bit-decomposition technique that utilizes the FeFET technology targeting multi-precision neural network acceleration. We explored different architecture blocks and presented optimizations required to maximize the architecture efficiency. Also, we showed the architecture operation as well as the mapping required for high utilization. Finally, we evaluated the performance of our architecture across several widely used neural network models and compared it to the state-of-the-art systems. By combining the FeFET technology and innovative structure for neural network acceleration, we achieved a peak performance of 36.5 TOPS/W at 8-bit/4-bit precision which can be seen as a huge improvement to the current state-of-the-art in-memory architectures. Further studies that consider applying various approximation innovations to the digital and analog blocks are yet to be explored.

REFERENCES

- [1] T. Ali, K. Kühnel, M. Czernohorsky, C. Mart, M. Rudolph, B. Pätzold, D. Lehninger, R. Olivo, M. Lederer, F. Müller, R. Hoffmann, J. Metzger, R. Binder, P. Steinke, T. Kämpfe, J. Müller, K. Seidel, and L. M. Eng. 2020. A study on the temperature-dependent operation of fluorite-structure-based ferroelectric HfO₂ memory FeFET: Pyroelectricity

- and reliability. *IEEE Transactions on Electron Devices* 67, 7 (2020), 2981–2987. DOI : <https://doi.org/10.1109/TED.2020.2995781>
- [2] Stefano Ambrogio, Prithvi Narayanan, Hsin-yu Tsai, Robert M. Shelby, Irem Boybat, Carmelo di Nolfo, Severin Sider, Massimo Giordano, Martina Bodini, Nathan C. P. Farinha, B. Killeen, C. Cheng, Y. Jaoudi, and G. W. Burr. 2018. Equivalent-accuracy accelerated neural-network training using analogue memory. *Nature* 558, 7708 (2018), 60–67.
 - [3] K. Ando, K. Ueyoshi, K. Orimo, H. Yonekawa, S. Sato, H. Nakahara, S. Takamaeda-Yamazaki, M. Ikebe, T. Asai, T. Kuroda, and M. Motomura. 2018. BRein memory: A single-chip binary/ternary reconfigurable in-memory deep neural network accelerator achieving 1.4 TOPS at 0.6 W. *IEEE Journal of Solid-State Circuits* 53, 4 (2018), 983–994.
 - [4] Shaahin Angizi, Zhezhi He, and Deliang Fan. 2018. DIMA: A depthwise CNN in-memory accelerator. In *Proceedings of the 2018 IEEE/ACM International Conference on Computer-Aided Design*. 1–8. DOI : <https://doi.org/10.1145/3240765.3240799>
 - [5] S. Beyer, S. Dünkler, M. Trentzsch, J. Müller, A. Hellmich, D. Utess, J. Paul, D. Kleimaier, J. Pellerin, S. Müller, J. Ocker, A. Benoist, H. Zhou, M. Mennenga, M. Schuster, F. Tassan, M. Noack, A. Pourkeramati, F. Müller, M. Lederer, T. Ali, R. Hoffmann, T. Kämpfe, K. Seidel, H. Mulaosmanovic, E. T. Breyer, T. Mikolajick, and S. Slesazeck. 2020. FeFET: A versatile CMOS compatible device with game-changing potential. In *Proceedings of the 2020 IEEE International Memory Workshop*. IEEE, 1–4.
 - [6] T. S. Böscke, J. Müller, D. Bräuhäus, U. Schröder, and U. Böttger. 2011. Ferroelectricity in hafnium oxide thin films. *Applied Physics Letters* 99, 10 (2011), 102903.
 - [7] Y. Cai, T. Tang, L. Xia, B. Li, Y. Wang, and H. Yang. 2020. Low bit-width convolutional neural network on RRAM. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 7 (2020), 1414–1427.
 - [8] X. Chen, X. Yin, M. Niemier, and X. S. Hu. 2018. Design and optimization of FeFET-based crossbars for binary convolution neural networks. In *Proceedings of the 2018 Design, Automation Test in Europe Conference Exhibition*. 1205–1210.
 - [9] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. 2016. *The cityscapes dataset for semantic urban scene understanding*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
 - [10] S. Cosemans, B. Verhoef, J. Doevenspeck, I. A. Papistas, F. Catthoor, P. Debacker, A. Mallik, and D. Verkest. 2019. Towards 10000TOPS/W DNN inference with analog in-memory computing – a circuit blueprint, device options and requirements. In *Proceedings of the 2019 IEEE International Electron Devices Meeting*. 22.2.1–22.2.4. DOI : <https://doi.org/10.1109/IEDM19573.2019.8993599>
 - [11] Cecilia De la Parra, Andre Guntoro, and Akash Kumar. 2020. ProxSim: GPU-based simulation framework for cross-layer approximate DNN optimization. In *Proceedings of the 2020 Design, Automation Test in Europe Conference Exhibition*. 1193–1198. DOI : <https://doi.org/10.23919/DATE48585.2020.9116476>
 - [12] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition*.
 - [13] L. Deng. 2012. The MNIST database of handwritten digit images for machine learning research [Best of the Web]. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142. DOI : <https://doi.org/10.1109/MSP.2012.2211477>
 - [14] S. Dünkler, M. Trentzsch, R. Richter, P. Moll, C. Fuchs, O. Gehring, M. Majer, S. Witte, B. Müller, T. Melde, H. Mulaosmanovic, S. Slesazeck, S. Müller, J. Ocker, M. Noack, D. LÄühr, P. Polakowski, J. Müller, T. Mikolajick, J. HÄüntschel, B. Rice, J. Pellerin, and S. Beyer. 2017. A FeFET based super-low-power ultra-fast embedded NVM technology for 22nm FDSOI and beyond. In *Proceedings of the 2017 IEEE International Electron Devices Meeting*. IEEE, 19.7.1–19.7.4.
 - [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE.
 - [16] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50× fewer parameters and <0.5MB model size. arXiv:1602.07360. Retrieved from <https://arxiv.org/abs/1602.07360>.
 - [17] Sambhav R. Jain, Albert Gural, Michael Wu, and Chris Dick. 2019. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. arXiv:1903.08066. Retrieved from <https://arxiv.org/abs/1903.08066>.
 - [18] P. Judd, J. Albericio, and A. Moshovos. 2017. Stripes: Bit-serial deep neural network computing. *IEEE Computer Architecture Letters* 16, 1 (2017), 80–83.
 - [19] A. Keshavarzi, K. Ni, W. Van Den Hoek, S. Datta, and A. Raychowdhury. 2020. FerroElectronics for edge intelligence. *IEEE Micro* 40, 6 (2020), 33–48. DOI : <https://doi.org/10.1109/MM.2020.3026667>
 - [20] H. Kim, Q. Chen, T. Yoo, T. T. Kim, and B. Kim. 2019. A 1-16b precision reconfigurable digital in-memory computing macro featuring column-MAC architecture and bit-serial computation. In *Proceedings of the ESSCIRC 2019 - IEEE 45th European Solid State Circuits Conference*. 345–348.

- [21] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. 2009. *CIFAR-10 (Canadian Institute for Advanced Research)*. Retrieved June 2021 from <http://www.cs.toronto.edu/~kriz/cifar.html>.
- [22] Y. Kwon and M. Rhu. 2018. Beyond the memory wall: A case for memory-centric HPC system for deep learning. In *Proceedings of the 2018 51st Annual IEEE/ACM International Symposium on Microarchitecture*. 148–161.
- [23] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86, 11 (1998), 2278–2324.
- [24] M. Lederer, T. Kämpfe, R. Olivo, D. Lehniger, C. Mart, S. Kirbach, T. Ali, P. Polakowski, L. Roy, and K. Seidel. 2019. Local crystallographic phase detection and texture mapping in ferroelectric Zr doped HfO₂ films by transmission-EBSD. *Applied Physics Letters* 115, 22 (2019), 222902.
- [25] M. Lederer, T. Kämpfe, T. Ali, F. Müller, R. Olivo, R. Hoffmann, N. Lalen, and K. Seidel. 2021. Ferroelectric field effect transistors as a synapse for neuromorphic application. *IEEE Transactions on Electron Devices* 68, 5 (2021), 2295–2300.
- [26] Maximilian Lederer, Franz Müller, Kati Kühnel, Ricardo Olivo, Konstantin Mertens, Martin Trentzsch, Stefan Dünkel, Johannes Müller, Sven Beyer, Konrad Seidel, Thomas Kämpfe, and Lukas M. Eng. 2020. Integration of hafnium oxide on epitaxial SiGe for p-type ferroelectric FET application. *IEEE Electron Device Letters* 41, 12 (2020), 1762–1765.
- [27] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H. Yoo. 2018. UNPU: A 50.6TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision. In *Proceedings of the 2018 IEEE International Solid - State Circuits Conference*. 218–220.
- [28] B. Li, Lixue Xia, Peng Gu, Y. Wang, and Huazhong Yang. 2015. MErging the interface: Power, area and accuracy co-optimization for RRAM crossbar-based mixed-signal computing system. In *Proceedings of the 2015 52nd ACM/EDAC/IEEE Design Automation Conference*. 1–6.
- [29] Yun Long, Daehyun Kim, Edward Lee, Priyabrata Saha, Burhan Ahmad Mudassar, Xueyuan She, Asif Islam Khan, and Saibal Mukhopadhyay. 2019. A ferroelectric FET-based processing-in-memory architecture for DNN acceleration. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 5, 2 (2019), 113–122. DOI : <https://doi.org/10.1109/JXCDC.2019.2923745>
- [30] Y. Long, T. Na, and S. Mukhopadhyay. 2018. ReRAM-based processing-in-memory architecture for recurrent neural network acceleration. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 26, 12 (2018), 2781–2794.
- [31] T. P. Ma and Jin-Ping Han. 2002. Why is nonvolatile ferroelectric memory field-effect transistor still elusive? *IEEE Electron Device Letters* 23, 7 (2002), 386–388. DOI : <https://doi.org/10.1109/LED.2002.1015207>
- [32] S. L. Miller and P. J. McWhorter. 1992. Physics of the ferroelectric nonvolatile memory field effect transistor. *Journal of Applied Physics* 72, 12 (1992), 5999–6010.
- [33] J. Müller, P. Polakowski, S. Mueller, and Thomas Mikolajick. 2015. Ferroelectric hafnium oxide based materials and devices: Assessment of current status and future prospects. *ECS Journal of Solid State Science and Technology* 4, 5 (2015), N30.
- [34] Borna Obradovic, Titash Rakshit, Ryan Hatcher, Jorge Kittl, Rwik Sengupta, Joon Goo Hong, and Mark S. Rodder. 2018. A multi-bit neuromorphic weight cell using ferroelectric FETs, suitable for SoC integration. *IEEE Journal of the Electron Devices Society* 6 (2018), 438–448. DOI : <https://doi.org/10.1109/JEDS.2018.2817628>
- [35] Andrew Howard, Andrey Zhmoginov, Liang-Chieh Chen, Mark Sandler, and Menglong Zhu. 2018. Inverted residuals and linear bottlenecks: Mobile networks for classification, detection and segmentation. *IEEE Conference on Computer Vision and Pattern Recognition* (2018), 4510–4520.
- [36] Satyabrata Sarangi and Bevan Baas. 2021. DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era. In *Proceedings of the 2021 IEEE International Symposium on Circuits and Systems*. 1–5. DOI : <https://doi.org/10.1109/ISCAS51556.2021.9401196>
- [37] A. Shafiee, A. Nag, N. Muralimohan, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *Proceedings of the 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture*. 14–26.
- [38] T. Soliman, F. Müller, T. Kirchner, T. Hoffmann, H. Ganem, E. Karimov, T. Ali, M. Lederer, C. Sudarshan, T. Kämpfe, A. Guntoro, and N. Wehn. 2020. Ultra-low power flexible precision FeFET based analog in-memory computing. In *Proceedings of the 2020 IEEE International Electron Devices Meeting*. 29.2.1–29.2.4. DOI : <https://doi.org/10.1109/IEDM13553.2020.9372124>
- [39] T. Soliman, R. Olivo, T. Kirchner, M. Lederer, T. Kämpfe, A. Guntoro, and N. Wehn. 2020. A ferroelectric FET based in-memory architecture for multi-precision neural networks. In *Proceedings of the 2020 33rd IEEE International System-on-Chip Conference*. IEEE.
- [40] T. Soliman, R. Olivo, T. Kirchner, C. D. I. Parra, M. Lederer, T. Kämpfe, A. Guntoro, and N. Wehn. 2020. Efficient FeFET crossbar accelerator for binary neural networks. In *Proceedings of the 2020 IEEE 31st International Conference on Application-specific Systems, Architectures and Processors*. 109–112.
- [41] L. Song, X. Qian, H. Li, and Y. Chen. 2017. PipeLayer: A pipelined ReRAM-based accelerator for deep learning. In *Proceedings of the 2017 IEEE International Symposium on High Performance Computer Architecture*. 541–552.

- [42] L. Wang, W. Kang, F. Ebrahimi, X. Li, Y. Huang, C. Zhao, K. L. Wang, and W. Zhao. 2018. Voltage-controlled magnetic tunnel junctions for processing-in-memory implementation. *IEEE Electron Device Letters* 39, 3 (2018), 440–443.
- [43] Huaqiang Wu, Xiao Hu Wang, Bin Gao, Ning Deng, Zhichao Lu, Brent Haukness, Gary Bronner, and He Qian. 2017. Resistive random access memory for future information processing system. *Proceedings of the IEEE* 105, 9 (2017), 1770–1789. DOI : <https://doi.org/10.1109/JPROC.2017.2684830>
- [44] H. Yan, H. R. Cherian, E. C. Ahn, X. Qian, and L. Duan. 2020. iCELIA: A full-stack framework for STT-MRAM-based deep learning acceleration. *IEEE Transactions on Parallel and Distributed Systems* 31, 2 (2020), 408–422.
- [45] Fisher Yu and Vladlen Koltun. 2015. Multi-Scale Context Aggregation by Dilated Convolutions. arXiv:1511.07122. Retrieved from <https://arxiv.org/abs/1511.07122>.
- [46] Shimeng Yu. 2017. *Neuro-inspired Computing Using Resistive Synaptic Devices*. Springer.
- [47] S. Yu. 2018. Neuro-inspired computing with emerging nonvolatile memories. *Proceedings of the IEEE* 106, 2 (2018), 260–285.
- [48] S. Yu, Z. Li, P. Chen, H. Wu, B. Gao, D. Wang, W. Wu, and H. Qian. 2016. Binary neural network with 16 Mb RRAM macro chip for classification and online training. In *Proceedings of the 2016 IEEE International Electron Devices Meeting*. 16.2.1–16.2.4.
- [49] W. Zhang, G. Wang, M. Tang, L. Cui, T. Wang, P. Su, Z. Chen, X. Long, Y. Xiao, and S. Yan. 2020. Impact of radiation effect on ferroelectric Al-doped HfO₂ metal-ferroelectric- insulator-semiconductor structure. *IEEE Access* 8 (2020), 108121–108126. DOI : <https://doi.org/10.1109/ACCESS.2020.3000865>
- [50] Qilin Zheng, Zongwei Wang, Zishun Feng, Bonan Yan, Yimao Cai, Ru Huang, Yiran Chen, Chia-Lin Yang, and Hai Helen Li. 2020. Lattice: An ADC/DAC-less ReRAM-based processing-in-memory architecture for accelerating deep convolution neural networks. In *Proceedings of the 2020 57th ACM/IEEE Design Automation Conference*. 1–6. DOI : <https://doi.org/10.1109/DAC18072.2020.9218590>
- [51] Z. Zhu, H. Sun, Y. Lin, G. Dai, L. Xia, S. Han, Y. Wang, and H. Yang. 2019. A configurable multi-precision CNN computing framework based on single bit RRAM. In *Proceedings of the 2019 56th ACM/IEEE Design Automation Conference*. 1–6.

Received 12 July 2021; revised 29 March 2022; accepted 30 March 2022